

Cyber Intrusion Detection — Project Files

Role of every file and folder in the repository.

Code

File / folder	Role
config.py	Central settings: paths, sequence shape (SEQ_LEN=15, NUM_FEATURES=78), label grouping, identifier columns to drop, training hyperparameters. Change behavior here, not in the scripts.
src/synthetic_data.py	Generates a CIC-IDS2017-shaped CSV (identifier columns, ~78 features, injected Inf/NaN, per-family distributions) so the pipeline runs before you download the real dataset. Test input, not a model or fake results.
src/preprocess.py	The data pipeline: merge CSVs, clean, drop identifiers, encode labels, build 15-flow sequences, split, fit the scaler on training data only, and save arrays, scaler, and label map.
src/models.py	Defines the two deep architectures: the 1D-CNN and the Transformer (with the PositionalEncoding layer). Run directly, it smoke-tests both.
src/train.py	Trains all three models (Random Forest baseline, then CNN and Transformer) with class weights, early stopping, and learning-rate reduction. Saves each trained model.
src/evaluate.py	Scores every model on the held-out test set: comparison table, per-class precision/recall/F1, attack false-positive rate, confusion matrix, PR curves. Builds the soft-vote ensemble.
src/__init__.py	Empty file marking src/ as a Python package so `python -m src.preprocess` works.
app/intrusion_dashboard.py	The Streamlit demo: loads saved models and scaler, takes a flow sequence (test sample, uploaded CSV, or random), shows the alert decision, confidence, and class probabilities.
tests/test_pipeline.py	Unit tests guarding the fragile parts: no leakage (scaler fit on train only), correct sequence shapes, identifiers dropped, valid probability outputs, and the FPR helper.
tests/__init__.py	Marks tests/ as a package.

Documentation and publishing

File / folder	Role
README.md	Project front page: what it is, why it's built this way, quickstart, the two diagrams, and the limitations section.
docs/PIPELINE_REFERENCE.md	Interview cheat sheet: every pipeline step mapped to the exact function and line, with a 'why' for each.
FILES.md	Markdown version of this same file-roles table.
requirements.txt	Python packages needed to run everything, with versions.
.gitignore	Tells Git what not to commit: the dataset, processed arrays, and trained models (all regenerable or too large).
LICENSE	MIT license.
assets/pipeline.svg	The pipeline data-flow diagram embedded in the README.
assets/ensemble.svg	The model-ensemble diagram embedded in the README.

Folders that hold inputs and outputs (not code)

File / folder	Role
data/raw/	Where you put the CIC-IDS2017 CSVs (or where the synthetic CSV lands). A .gitkeep keeps the empty folder in Git.
data/processed/	Where preprocessing saves the .npz arrays (X_train, y_train, etc.).
models/	Where trained models (cnn_model.keras, transformer_model.keras, rf_baseline.pkl), the fitted scaler.pkl, and label_map.json are saved.
reports/	Where evaluation writes its output: metrics.json, results.md, confusion_matrix.png, pr_curves.png, training_history.json.

How to remember the layout: config.py plus the four files in src/ are the pipeline (settings → data → models → training → evaluation). app/ is the demo. tests/, docs/, and the root files are the publishing and quality layer. The data, models, and reports folders just hold inputs and outputs.